

cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h` . You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

Notation	Meaning
<Species>	Ion , Alpha , Beam
<MHDField>	Phi , A , dNe , dTe
<PICField>	dPi , dPa , dPb

Data Types and File Paths

Parameter	Allowed Values	Description	Notes	Example
mhdReal	float / double	MHD calculation precision.	MHD precision should be greater than or equal to PIC precision.	using mhdReal = double;
picReal	float / double	PIC calculation precision.	PIC precision should be less than or equal to MHD precision.	using picReal = float;
inputDir	String path	Input file directory.	For a run from scratch, only the MHDCollocated file and the optional MHDStaggered file are needed. For a continued run, the MHDContinue and PICContinue files are also needed. They come from the previous run's outputDir/final .	const std::string inputDir = "/path/to/input";
outputDir	String path	Output file directory.		const std::string outputDir = "/path/to/output";
MHDCollocated	.bin file name	Collocated MHD equilibrium file.		const std::string MHDCollocated = "MHDCollocated.bin";
MHDStaggered	.bin file name	Staggered MHD equilibrium file.	Required only when ifStaggered=trueType .	const std::string MHDStaggered = "MHDStaggered.bin";
<Species>PhaseSpaceMapping	.bin file name	Phase-space orbit mapping file.	Required only when ifOutputPhaseSpaceOrbit=trueType .	const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";

Normalization Parameters

Parameter	Allowed Values	Description	Notes	Example
B0	Positive real number	Magnetic-field normalization scale.		const double B0 = 4.921751144619735;
L0	Positive real number	Length normalization scale.		const double L0 = 6.595629295925759;
VA0	Positive real number	Velocity normalization scale.		const double VA0 = 8.864164667700194e+06;
RH00	Positive real number	Left endpoint of the radial interval.		const double RH00 = 0.08;
RH01	Positive real number	Right endpoint of the radial interval.		const double RH01 = 0.90;
PSITMAX	Positive real number	Outermost toroidal magnetic flux (Wb/rad).		const double PSITMAX = 18.868213504765762;

Grid and Parallel Scale

Parameter	Allowed Values	Description	Notes	Example
hostNums	Positive integer	Number of MPI processes.		const int hostNums = 4;
devNums	Positive integer	Number of GPUs used by each MPI process.		const int devNums = 4;
gridNx	Positive integer	Number of radial grid points.		const int gridNx = 256;
gridNy	Positive integer	Number of field-aligned grid points.	Must be divisible by hostNums * devNums .	const int gridNy = 32;
gridNz	Positive integer	Number of toroidal grid points.		const int gridNz = 96;
ppcNums	Positive integer	Average number of particles per species in each grid cell.		const int ppcNums = 256;

Toroidal Range and Initial Perturbation

Parameter	Allowed Values	Description	Notes	Example
tubes	Positive integer	Simulate $1/\text{tubes}$ of the toroidal domain.		<code>const int tubes = 6;</code>
leftN	Nonnegative integer	Lower bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.	The actual toroidal mode range is <code>tubes*[leftN,rightN]</code> , with spacing <code>tubes</code> .	<code>const int leftN = 1;</code>
rightN	Nonnegative integer	Upper bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int rightN = 6;</code>
refinedTimes	Positive integer	Field-aligned interpolation refinement factor used by <code>selectNM_*</code> poloidal filtering.	The refined grid size is <code>gridNy * refinedTimes</code> .	<code>const int refinedTimes = 32;</code>
perturbLeftN	Nonnegative integer	Lower bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.	The actual toroidal mode range is <code>tubes*[perturbLeftN,perturbRightN]</code> , with spacing <code>tubes</code> .	<code>const int perturbLeftN = 1;</code>
perturbRightN	Nonnegative integer	Upper bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int perturbRightN = 6;</code>
perturbRadialIndex	1 to <code>gridNx</code>	Radial peak grid point of the initial Gaussian perturbation.		<code>const int perturbRadialIndex = 85;</code>
perturbWidth	Positive real number	Radial width of the initial Gaussian perturbation.		<code>const mhdReal perturbWidth = 0.04;</code>
perturbAmplitude	Real number	Normalized <code>Phi</code> amplitude of the initial Gaussian perturbation.		<code>const mhdReal perturbAmplitude = 2.5e-9;</code>

Filtering

Parameter	Allowed Values	Description	Notes	Example
ifFilterN_MHDField	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the MHD field.		<code>using ifFilterN_Phi = trueType;</code>
ifFilterN_dP	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifFilterN_dP = trueType;</code>
removeN_MHDField	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the MHD field.		<code>constexpr std::array<int, 1> removeN_Phi = {7};</code>
removeN_dP	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<int, 0> removeN_dP = {};</code>
selectNM_MHDField	<code>{{N, leftM, rightM}, ...}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the MHD field.		<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{{0, 0, 0}}};</code>
selectNM_dP	<code>{{N, leftM, rightM}, ...}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{{0, 0, 1}}};</code>

MHD Physics Switches

Parameter	Allowed Values	Description	Notes	Example
ifStaggered	<code>trueType</code> / <code>falseType</code>	Whether to include the staggered grid.	Requires the <code>MHDStaggered</code> file.	<code>using ifStaggered = falseType;</code>
ifNonlinearMHD	<code>trueType</code> / <code>falseType</code>	Whether to include MHD nonlinear terms.		<code>using ifNonlinearMHD = trueType;</code>
ifEparallel	<code>trueType</code> / <code>falseType</code>	Whether to include the parallel electric-field term.		<code>using ifEparallel = trueType;</code>
ifFLRMHD	<code>trueType</code> / <code>falseType</code>	Whether to include the FLR effect from background thermal ions in the Poisson equation.		<code>using ifFLRMHD = falseType;</code>
ifQNeutrality	<code>trueType</code> / <code>falseType</code>	Whether to compute the electron density perturbation from quasi-neutrality.	When enabled, <code>dNe</code> is determined by the vorticity term and perturbed densities of enabled species.	<code>using ifQNeutrality = trueType;</code>
ifMaxwellStress	<code>trueType</code> / <code>falseType</code>	Whether to include Maxwell stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifMaxwellStress = trueType;</code>
ifReynoldsStress	<code>trueType</code> / <code>falseType</code>	Whether to include Reynolds stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifReynoldsStress = trueType;</code>
MaxwellStressCoef	Real number	Maxwell stress coefficient.	Effective only when <code>ifMaxwellStress=trueType</code> .	<code>const mhdReal MaxwellStressCoef = 1.0;</code>
ReynoldsStressCoef	Real number	Reynolds stress coefficient.	Effective only when <code>ifReynoldsStress=trueType</code> .	<code>const mhdReal ReynoldsStressCoef = 1.0;</code>

Dissipation

Parameter	Allowed Values	Description	Notes	Example
ifNablaPerp2_MHDField	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular MHD-field dissipation.		<code>using ifNablaPerp2Phi = trueType;</code>
perp2_MHDField	Positive real number	Coefficient for second-order perpendicular MHD-field dissipation.		<code>const mhdReal perp2Phi = 1.0e-7;</code>
ifNablaPara4_MHDField	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel MHD-field dissipation.		<code>using ifNablaPara4Phi = trueType;</code>
para4_MHDField	Positive real number	Coefficient for fourth-order parallel MHD-field dissipation.		<code>const mhdReal para4Phi = 1.0e-6;</code>
ifNablaPerp2dP	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPerp2dP = trueType;</code>
perp2dP	Positive real number	Coefficient for second-order perpendicular PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>const mhdReal perp2dP = 1.0e-6;</code>
ifNablaPara4dP	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPara4dP = falseType;</code>
para4dP	Positive real number	Coefficient for fourth-order parallel PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>const mhdReal para4dP = 0.0;</code>

PIC Physics Switches

Parameter	Allowed Values	Description	Notes	Example
ifFLRPIC	trueType / falseType	Whether to enable the FLR effect in the PIC part.		using ifFLRPIC = trueType;
ifNonlinearPIC	trueType / falseType	Whether to enable PIC nonlinear terms.		using ifNonlinearPIC = trueType;
gyroNums	Positive integer	Number of gyro-average points.		const int gyroNums = 4;

PIC Species Parameters

Parameter	Allowed Values	Description	Notes	Example
if<Species>	trueType / falseType	Whether to enable the corresponding ion species.		using ifIon = trueType;
<Species>Type	Maxwell / Slowing0 / Slowing1 / Slowing2 / Slowing3	Velocity distribution type.		const distType IonType = Maxwell;
<Species>Space	spaceReal / spaceUniform	Spatial sampling method: marker positions follow the physical spatial distribution or a uniform distribution.		const spaceType IonSpace = spaceReal;
<Species>Velocity	velocityReal / velocityUniform	Velocity sampling method: marker velocities follow the physical velocity-space distribution or a uniform distribution.		const velocityType IonVelocity = velocityUniform;
<Species>Mass	Positive real number	Ion mass, normalized by the proton mass.		const picReal IonMass = 2.5;
<Species>Char	Positive real number	Ion charge, normalized by the electron charge.		const picReal IonChar = 1.0;
<Species>Beta	Positive real number	Ion beta on the magnetic axis ($P/(B0^2/(2*\mu0))$).		const picReal IonBeta = 0.037793721898356;
<Species>Vmin	Positive real number	Lower bound of the ion velocity, normalized by $VA0$.		const picReal IonVmin = 0.0135;
<Species>Vmax	Positive real number	Upper bound of the ion velocity, normalized by $VA0$.		const picReal IonVmax = 0.54;
<Species>Vb	Positive real number	Cutoff velocity of the slowing-down distribution.		const picReal BeamVb = 0.1;
<Species>DeltaV	Positive real number	Cutoff width of the slowing-down distribution.		const picReal BeamDeltaV = 0.1;
<Species>Lambda0	Positive real number	Center of Λ in the anisotropic distribution.		const picReal BeamLambda0 = 0.4;
<Species>DeltaLambda2	Positive real number	Square of the Λ width in the anisotropic distribution.		const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);

Distribution Type	Formula
Maxwell	$f \sim n * T^{(-3/2)} * \exp[-m*v^2/(2*T)]$
Slowing0	$f \sim n / (v^3 + v_c^3)$
Slowing1	$f \sim n * [1 + \operatorname{erf}((V_b - v)/\Delta v)] / (v^3 + v_c^3)$
Slowing2	$f \sim n * \exp[-(\Lambda - \Lambda_0)^2 / \Delta \Lambda^2] / (v^3 + v_c^3)$
Slowing3	$f \sim n * [1 + \operatorname{erf}((V_b - v)/\Delta v)] * \exp[-(\Lambda - \Lambda_0)^2 / \Delta \Lambda^2] / (v^3 + v_c^3)$

Diagnostic Switches

Parameter	Allowed Values	Description	Notes	Example
<code>ifDiagAmplitude</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode amplitude.		<code>using ifDiagAmplitude = trueType;</code>
<code>ifDiagFrequency</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode frequency.		<code>using ifDiagFrequency = trueType;</code>
<code>ifDiagEparallel</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the parallel electric field.		<code>using ifDiagEparallel = trueType;</code>
<code>ifDiagDensity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose ion perturbed density.		<code>using ifDiagDensity = trueType;</code>
<code>ifDiagDiffusivity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose ion diffusivity.		<code>using ifDiagDiffusivity = trueType;</code>
<code>ifDiagZFDrive</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the zonal-flow drive source.		<code>using ifDiagZFDrive = falseType;</code>
<code>ifCheckNaN</code>	<code>trueType</code> / <code>falseType</code>	Whether to check for NaN.	If NaN is diagnosed, the program stops immediately and writes output.	<code>using ifCheckNaN = trueType;</code>

MHD Field Output Switches

Parameter	Allowed Values	Description	Notes	Example
<code>ifOutputPhi</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>Phi</code> at multiple time points.		<code>using ifOutputPhi = trueType;</code>
<code>ifOutputA</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>A</code> at multiple time points.		<code>using ifOutputA = falseType;</code>
<code>ifOutputdNe</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dNe</code> at multiple time points.		<code>using ifOutputdNe = falseType;</code>
<code>ifOutputdTe</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dTe</code> at multiple time points.		<code>using ifOutputdTe = falseType;</code>
<code>ifOutputdPi</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPi</code> at multiple time points.		<code>using ifOutputdPi = falseType;</code>
<code>ifOutputdPa</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPa</code> at multiple time points.		<code>using ifOutputdPa = falseType;</code>
<code>ifOutputdPb</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPb</code> at multiple time points.		<code>using ifOutputdPb = falseType;</code>

Phase-space Output Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>gridE</code>	Positive integer	Number of phase-space grid points in the <code>E</code> direction.		<code>const int gridE = 96;</code>
<code>gridPphi</code>	Positive integer	Number of phase-space grid points in the <code>Pphi</code> direction.		<code>const int gridPphi = 128;</code>
<code>gridLambda</code>	Positive integer	Number of phase-space grid points in the <code>Lambda</code> direction.		<code>const int gridLambda = 48;</code>
<code>ppcPhase</code>	Positive integer	Average number of particles per species in each phase-space grid cell.		<code>const int ppcPhase = 2048;</code>
<code>ifOutputPhaseSpaceJacobian</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space Jacobian.	Output during the initialization stage.	<code>using ifOutputPhaseSpaceJacobian = trueType;</code>
<code>ifOutputPhaseSpaceOrbit</code>	<code>trueType</code> / <code>falseType</code>	Whether to output phase-space orbit frequencies.	Requires the <code><Species>PhaseSpaceMapping</code> file.	<code>using ifOutputPhaseSpaceOrbit = falseType;</code>
<code>ifOutputPhaseSpaceF0</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space equilibrium distribution function.	Output during the initialization stage.	<code>using ifOutputPhaseSpaceF0 = falseType;</code>
<code>ifOutputPhaseSpaceDeltaF</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space perturbed distribution function.		<code>using ifOutputPhaseSpaceDeltaF = falseType;</code>
<code>ifOutputPhaseSpacePower</code>	<code>trueType</code> / <code>falseType</code>	Whether to output phase-space wave-particle interaction power.		<code>using ifOutputPhaseSpacePower = falseType;</code>
<code>gridVpara</code>	Positive integer	Number of pitch-space grid points in the <code>vpara</code> direction.		<code>const int gridVpara = 128;</code>
<code>gridVperp</code>	Positive integer	Number of pitch-space grid points in the <code>vperp</code> direction.		<code>const int gridVperp = 64;</code>
<code>ppcPitch</code>	Positive integer	Average number of particles per species in each pitch-space grid cell.		<code>const int ppcPitch = 2048;</code>
<code>ifOutputPitchSpaceJacobian</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space Jacobian.	Output during the initialization stage.	<code>using ifOutputPitchSpaceJacobian = trueType;</code>
<code>ifOutputPitchSpaceF0</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space equilibrium distribution function.	Output during the initialization stage.	<code>using ifOutputPitchSpaceF0 = falseType;</code>
<code>ifOutputPitchSpaceDeltaF</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space perturbed distribution function.		<code>using ifOutputPitchSpaceDeltaF = falseType;</code>
<code>ifOutputPitchSpacePower</code>	<code>trueType</code> / <code>falseType</code>	Whether to output pitch-space wave-particle interaction power.		<code>using ifOutputPitchSpacePower = falseType;</code>

Time Advancement

Parameter	Allowed Values	Description	Notes	Example
<code>ifContinue</code>	<code>trueType</code> / <code>falseType</code>	Whether to start from continue files.	The <code>inputDir</code> directory must contain the <code>MHDContinue</code> and <code>PICContinue</code> continue files. They come from the previous run's <code>outputDir/final</code> .	<code>using ifContinue = falseType;</code>
<code>continueSteps</code>	Nonnegative integer	Number of steps already completed before resuming.	Must match the suffix of the continue file names.	<code>const int continueSteps = 0;</code>
<code>dt</code>	Positive real number	MHD time step.		<code>const double dt = 0.02;</code>
<code>totalSteps</code>	Positive integer	Total number of MHD steps in this run.		<code>const int totalSteps = 20000;</code>
<code>ratioDt</code>	Positive integer	Ratio of the PIC time step to the MHD time step.	PIC uses <code>dt * ratioDt</code> for each push.	<code>const int ratioDt = 1;</code>
<code>sortSteps</code>	Positive integer	Particle sorting interval.		<code>const int sortSteps = 25;</code>
<code>diagSteps</code>	Positive integer	Diagnostic sampling interval.		<code>const int diagSteps = 1;</code>
<code>outputSteps</code>	Positive integer	Field and phase-space output interval.		<code>const int outputSteps = 2500;</code>